



Authentication and Identity Management



The OpenShift OAuth Server and Identity Providers

Chapter 1 Section 1



Objectives

After completing this section, you will be:

- Define the concepts and custom resources
 - of the OpenShift OAuth server
- Understand how these
 - resources augment Kubernetes authentication.

Authentication, Authorization and Accounting

Authentication:

- Verifies who a user is through credentials like a password, certificate, or other biometric data.
- It answers the question, "Who are you?".

Authorization:

- Determines the specific resources, services, or actions a user is allowed to access after their identity has been confirmed.
- It answers the question, "What can you do?".

Accounting:

- Logs user or admin activity during their session, including the resources they accessed, how long they were active, and any commands they issued.
- This information is used for auditing and can be used for billing or capacity planning.

Authentication methods

Authenticating API Requests

- Simple form of authentication.
- Using API Keys in header or as a query parameter.
- Case use: Third party application accessing Kubernetes resources.

Authentication in Kubernetes

- Core component are API server and authentication plug-ins.
- Can use username/password or token or client certificate.
- Case use : Users to access and manage resources in Kubernetes.

Authentication in OpenShift

- Include all method in Kubernetes.
- Extend Kubernetes authentication:
 - Built-in OAuth server.
 - Multiple authenticated method.
 - Manage conflicting authentication.

Authentication in OpenShift

X.509 client certificates:

- Validates client certificates against trusted certificate authority.
- Use HTTPS connection to API server.

OAuth access tokens:

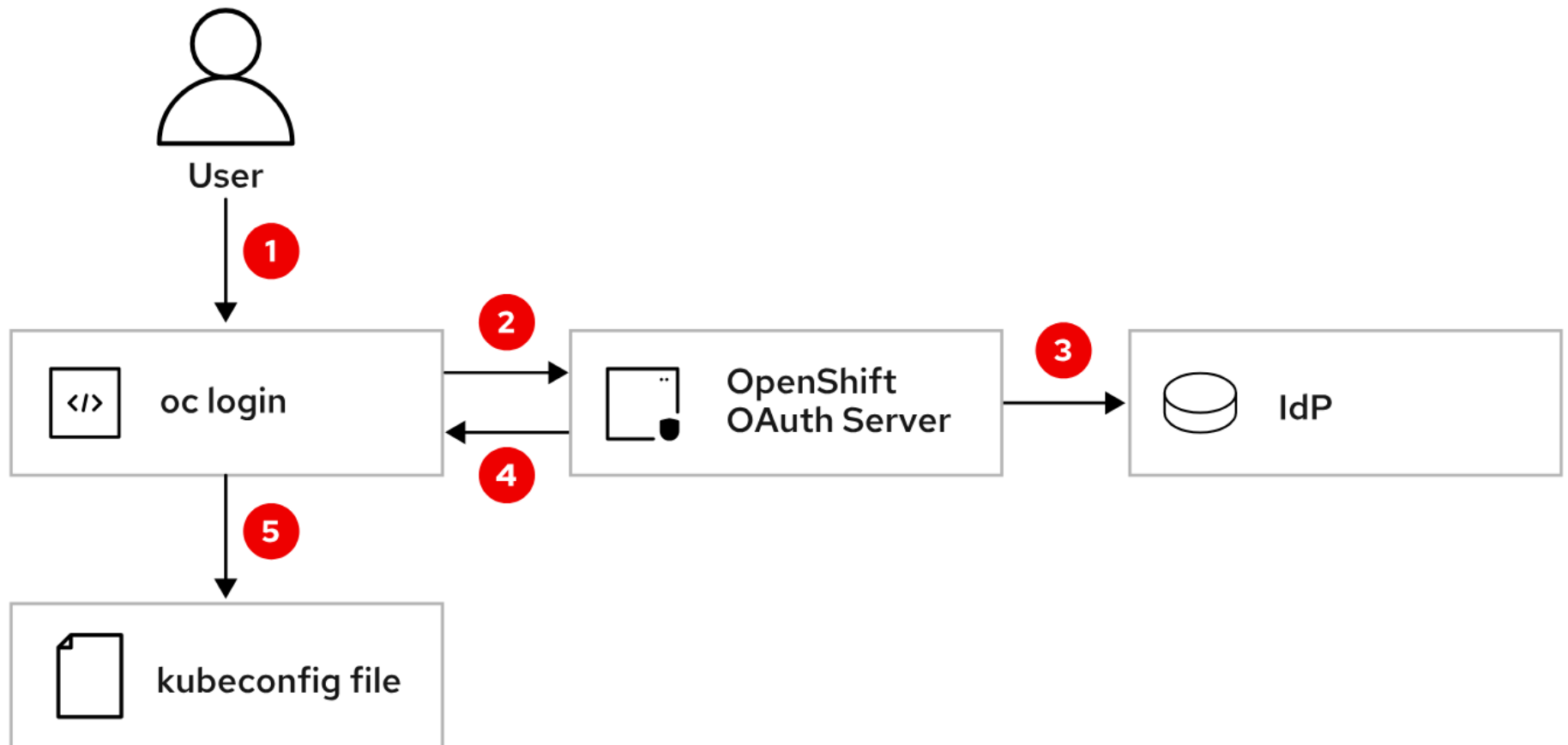
- Built-in and default authentication operator in OCP.
- Issue access tokens to users after authentication.
- Support various identity providers (IdPs)
 - Apache httpasswd
 - GitHub & GitLab credential
 - Lightweight Directory Access Protocols (LDAP)
 - OpenStack Keystone
 - OpenID Connect

IdP used in lab environment - comparison

Feature	Htpasswd	LDAP	OIDC
Type	File-based	Centralized directory service	Identity layer on top OAuth
Primary Use	Development, testing, basic auth for simply static site	Enterprise user/group management (Active Directory, OpenLDAP)	Modern web/mobile app SSO and identity verification
Complexity	Very low	High	Moderate
Scalability	Poor (limited by file size and manual management)	High (designed for large enterprise networks)	High (designed for cloud-scale applications)
Security	Basic (uses simple hashing, insecure for production use)	Robust (supports various auth methods including PKI, SASL)	Robust (uses modern cryptography, tokens)
User Info	Stores only username and hashed password	Stores rich user attributes (email, groups, POSIX attributes, etc.)	Provides user claims via ID tokens (user ID, email, name, etc.)

The OpenShift OAuth Server

- Built-in and default authentication.
- Generate tokens for authenticated users.
- Interact with `oc login` command.



Set logging level during login

- **oc login -u user -p password https://api.ocp4.example.com:6443 --loglevel 7**

10929 ...] HEAD **https://api.ocp4.example.com:6443/**

10929 ...] Request Headers:

10929 ...] Response Status: 403 Forbidden in 20 milliseconds

10929 ...] GSSAPI Enabled

10929 ...] GET **https://api.ocp4.example.com:6443/.well-known/oauth-authorization-server**

10929 ...] Request Headers:

10929 ...] X-Csrf-Token: 1

10929 ...] Response Status: 200 OK in 0 milliseconds

10929 ...] using system roots as no error was encountered

10929 ...] **GET https://oauth-**

openshift.apps.ocp4.example.com/oauth/authorize?client_id=openshift-challenging-client...

...output omitted...

Post authentication

- Each time `oc` cmd execute:
 1. OCP reads credentials from kubeconfig file.
 2. OCP uses credentials from kubeconfig file to authenticate to API Server



User

1

2



kubeconfig file



oc command

3



Kubernetes
API server

Configure the OpenShift OAuth Server

- Mandatory : One IdP in OAuth server
- Query current OAuth object in OCP

```
$ oc get oauth cluster -o yaml
```

```
apiVersion: config.openshift.io/v1
```

```
kind: OAuth
```

```
...output omitted ...
```

```
spec:
```

```
  identityProviders:
```

```
    - ldap:
```

```
...output omitted...
```

```
  bindDN: uid=admin,cn=users,cn=accounts,dc=ocp4,dc=example,dc=com
```

```
  bindPassword: ldap://idm.ocp4.example.com/cn=users,cn=accounts,dc=ocp4,...
```

```
  mappingMethod: claim
```

```
  name: Red Hat Identity Management
```

```
  type: LDAP
```

Using httpasswd IdP

```
apiVersion: config.openshift.io/v1
```

```
kind: OAuth
```

```
metadata:
```

```
  name: cluster
```

```
spec:
```

```
  identityProviders:
```

```
    - name: httpasswd-idp-name
```

```
      mappingMethod: claim 3
```

```
      type: HTTPasswd
```

```
      httpasswd:
```

```
        fileData:
```

```
          name: httpasswd-secret
```

```
    - name: ldap-idp-name
```

```
      mappingMethod: claim
```

```
      type: LDAP
```

```
      ldap:
```

```
...output omitted...
```

Users, Groups, and Identities

- By default, Kubernetes does not have resources represent these.
- K8s uses certificates or tokens.
- Identity Providers (IdP) can creates:

Object	Description
User accounts	Able to login with password, then get assign with access token
Groups	To simplify permission assignments
Identities	Record of successful authentication attempts from user accounts
Service accounts	Link pod or other resource to interact with other k8s resources such as routes, security object, external objects and etc
System groups	Predefined groups to grant specific permissions to selective users. system:authenticated - represent all authenticated users system:unauthenticated - represent all un-authenticated users. Not yet pass any authentication process. system:masters - users with administrative access

OAuth Server Mapping Methods

- On first login, OCP
 - Sync user account information from IdP,
 - Create identity mapped to user account.
- When multiple different IdPs are configured:
 - Use mappingMethod to manage collisions
 - Establish mappings between IdP
- mappingMethod configurable values:
 - **claim:**

Is default. Preferred username. May fail due to redundant username from another identity. Best use when only single IdP is used.
 - **add:**

If username exists in OCP, maps to existing user and add new identity to existing mapping. Best use to resolve conflicting multiple IdP.
 - **lookup:**

Does not automatically provision user or identities
Use this method for manually provision or use external process

Authentication on New OpenShift Cluster

- Initial installation, OCP creates:
 1. `kubeadmin` **user with password**.
 2. `system:admin` **user with client certificate in kubeconfig file**.
 3. **Assign both user to cluster-role**.
- Recommendation:
 - **Configure IdP with customized administrator accounts and**
 - **Delete kubeadmin user.**
 - **Store the system:admin certificate in safe place for later recovery purposes.**

...output omitted...

INFO Install complete!

INFO To access the cluster as **the system:admin user** when using 'oc', run 'export KUBECONFIG=**/home/lab/ocp4/auth/kubeconfig**'

INFO Access the OpenShift web-console here: <https://console-openshift-console.apps.ocp4.example.com>

INFO Login to the console with **user: "kubeadmin"**, and **password: "iMcm5-eCy2Q-PH66E-NkyPN"**

INFO Time elapsed: 10m52s

...output omitted...

Deleting kubeadmin user

- Cannot be undone.
- Make sure to configure IdP with cluster-role administrator accounts.
- To remove kubeadmin from cluster; login as administrator user
`$ oc login -u admin -p password api.ocp4.example.com:6443`
<enter admin's password>

`$ oc delete secrets kubeadmin -n kube-system`

QUIZ: The OpenShift OAuth Server and Identity Providers

Section 1.2

5 questions

Quiz – 1 of 5

Which authentication method handles user authentication in OpenShift, and is not present in Kubernetes?

- a) Client certificates
- b) OAuth server
- c) Internal Certificate Authority
- d) LDAP

Quiz – 1 of 5

Which authentication method handles user authentication in OpenShift, and is not present in Kubernetes?

- a) Client certificates
- b) OAuth server**
- c) Internal Certificate Authority
- d) LDAP

Quiz – 2 of 5

Which is the cluster administrator user that the OpenShift installation process creates?

- a) Root
- b) Admin
- c) Kubeadmin
- d) k8sadmin

Quiz – 2 of 5

Which is the cluster administrator user that the OpenShift installation process creates?

- a) Root
- b) Admin
- c) **Kubeadmin**
- d) k8sadmin

Quiz – 3 of 5

Which three API resources are API extension resources that OpenShift OAuth defines? (Choose three.)

- a) Users
- b) Identities
- c) Roles
- d) Groups
- e) Role bindings

Quiz – 3 of 5

Which three API resources are API extension resources that OpenShift OAuth defines? (Choose three.)

- a) **Users**
- b) **Identities**
- c) Roles
- d) **Groups**
- e) Role bindings

Quiz – 4 of 5

Your company configures two IdPs, httpasswd and LDAP, in the internal OpenShift OAuth server. Because both IdPs have some users in common, you must configure the mapping method parameter in the OAuth server to avoid any collision between user identities. Which mapping method maps the identities from both IdPs to the same user and avoids collisions?

- a) claim
- b) add
- c) lookup
- d) No method can avoid collisions

Quiz – 4 of 5

Your company configures two IdPs, httpasswd and LDAP, in the internal OpenShift OAuth server. Because both IdPs have some users in common, you must configure the mapping method parameter in the OAuth server to avoid any collision between user identities. Which mapping method maps the identities from both IdPs to the same user and avoids collisions?

- a) claim
- b) add**
- c) lookup
- d) No method can avoid collisions

Quiz – 5 of 5

In OpenShift, how does the OAuth server handle user authentication?

- a) Relies on client certificates that the IdP issues.
- b) Delegates authentication to the IdP and generates a user OAuth token after the IdP validates the user credentials.
- c) Uses OAuth tokens that the IdP generates.

Quiz – 5 of 5

In OpenShift, how does the OAuth server handle user authentication?

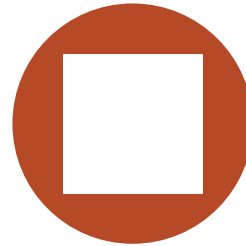
- a) Relies on client certificates that the IdP issues.
- b) Delegates authentication to the IdP and generates a user OAuth token after the IdP validates the user credentials.**
- c) Uses OAuth tokens that the IdP generates.

Section Summary

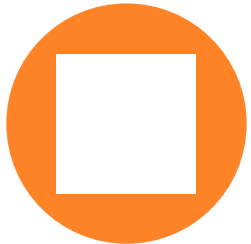
In this section, you learned:



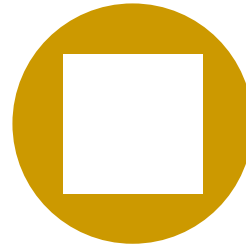
Understand and Configure OAuth Server



Understand Identity Providers(IdP)



Understand Users, Groups and Identities



Configure OAuth Server Mapping Method

LDAP Authentication and Group Synchronization

Chapter 1 Section 3

Single Sign On

- Need for SSO
 - Custom cluster accounts and groups.
 - Provide centralized authentication to:
 - Server (File/Print/Database/Web and etc)
 - Storage controllers.
 - Network appliances.
 - Supported software, hardware, software-defined
 - On-prem or cloud or hybrid infrastructure.
- OCP/K8s post installation uses local user - the kubeadmin

The LDAP Service

- LDAP provides SSO.
- Lightweight Directory Access Protocol.
- Standard protocol for
 - Querying system to authenticate users and groups.
 - Centralize management of network resources (network devices, storage systems)
 - Search information from directory service. (file server/print server)
- Integrates third-party directory services:
 - Red Hat Identity Management (IdM)
 - Microsoft Active Directory (AD)
 - Microsoft Entra ID (Azure AD)

Binding OAuth to LDAP



LDAP – provides user authentication information

OAuth – create access token

RBAC – bind token to respective API resources through role assignment



Module does not cover LDAP administration in depth,
only necessary information for integrating LDAP in OCP cluster.

Authentication by Using LDAP

- Validate user-provided login credentials for the cluster.
- Query for information in LDAP database.

- Uses URI and standard syntax for interaction.

```
ldap://host:port/basedn?attribute?scope?filter
```

- ldap – protocol. If LDAP over SSL, uses ldaps
 - host:port – LDAP server name and listening port. Default localhost:389
 - basedn – directory where query begins. DN is distinguished name
 - attributes – target for search
 - scope – subscription area for the lookup.
 - filter – refines results for the query. Defaults to (objectClass=*)
- Example: lookup for specific username in the LDAP server:

```
ldap://ldap.example.com:389/dc=example,dc=com?givenName,sn,cn?(uid=john.smith)
```

LDAP CLI Queries

- Use `ldapsearch` tool to inspect and validate available LDAP connectivity.

```
$ ldapsearch -x -D "cn=administrator,dc=www,dc=example,dc=com" -W \  
-H ldap://ldap.example.com -b "uid=john.smith" "objectclass=account" uid
```

Parameter	Description
-x	Use simple authentication
-D bind_dn	The distinguished name to administrator account
-H	Protocol and server host name
-b	Search base or root location for the lookup
-W	Prompt for password

- Organizational units - describe the path to the correct location for an information lookup within the LDAP system:

```
dn: uid=john.smith,ou=users,dc=www,dc=example,dc=com
```



Module does not cover LDAP administration in depth,
only necessary information for integrating LDAP in OCP cluster.

Configuring LDAP as OpenShift IdP

1. Create secret containing sensitive information:
 - Password to LDAP server.
2. Create config map containing insensitive information
 - Certificate authority bundles.
3. Update OAuth cluster resource
 - mappingMethod
 - preferredUsername
 - bindDN
 - bindPassword
 - CA

Create the bindPassword Secret Resource

- The following example is a YAML file for creating the bindPassword secret for an LDAP IdP:

```
apiVersion: v1
kind: Secret
metadata:
  name: ldap-secret
  namespace: openshift-config
type: Opaque
data:
  bindPassword: base64-encoded-bind-password
```

- Ensure secret is created in `openshift-config` namespace.
- Use base64-encoded format.

Create the Certificate Configuration Map

- The following example is a YAML file that defines how to create a configuration map resource for the certificate bundle:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: ca-config-map
  namespace: openshift-config
data:
  ca.crt: |
    CA_certificate_PEM
```

- Ensure configMap is created in `openshift-config` namespace.
- Define the certificate authority bundles.

Update the OAuth CR (1)

- Login with cluster-admin role and modify the oauth cluster resource:

\$ oc edit oauth cluster

```
apiVersion: config.openshift.io/v1
```

```
kind: OAuth
```

```
metadata:
```

```
  name: cluster
```

```
spec:
```

```
  identityProviders:
```

```
    - name: ldapidp
```

```
      mappingMethod: claim
```

```
      type: LDAP
```

```
      ldap:
```

```
        attributes:
```

```
          id:
```

```
            - dn
```

```
          email:
```

```
            - mail
```

```
          name:
```

```
            - cn
```

```
          preferredUsername:
```

```
            - uid
```

```
Continue >
```

Update the OAuth CR (2)

- The following example shows the configuration for defining an LDAP IdP in the OAuth CR YAML file::

```
apiVersion: config.openshift.io/v1
```

```
kind: OAuth
```

```
metadata:
```

```
  name: cluster
```

```
spec:
```

```
  identityProviders:
```

```
  - name: ldapidp
```

```
    mappingMethod: claim
```

```
    type: LDAP
```

```
    ldap:
```

```
      attributes:
```

```
        id:
```

```
        - dn
```

```
        email:
```

```
        bindDN: 'cn=admin,dc=example,dc=com'
```

```
        bindPassword:
```

```
          name: ldap-secret
```

```
        ca:
```

```
          name: ca-config-map
```

```
        insecure: false
```

```
        url: "ldaps://ldaps.example.com/ou=users,dc=acme,dc=com?uid"
```


OpenShift Group Synchronization with LDAP

- User accounts – for authentication.
- Groups – for authorization / simplifies permission assignments.
- With LDAP, business manage group memberships in single location.
- Synchronize groups from LDAP into OpenShift cluster.
- Requires following:
 - LDAP Sync configuration file
 - Query definition

LDAP Sync Configuration file

- Example of LDAP sync configuration:

```
url: ldap://1.2.3.4:389  
bindDN: cn=administrator,dc=example,dc=com  
bindPassword: password  
insecure: false  
ca: ca-bundle.crt
```

- Connection parameters:
 - url – protocol + hostname + listening port
 - bindDN – administrator account in LDAP directory service
 - bindPassword – pointing to secret
 - Insecure: false – using TLS/SSL. If set to true then not using TLS/SSL.
 - ca – certificate bundles

LDAP Query Definition

- Example of LDAP query definition:

```
baseDN: ou=groups,dc=example,dc=com  
scope: sub  
timeout: 0  
filter: (objectClass=group)  
pageSize: 0
```

- Parameters:
 - baseDN – organization unit + fqdn
 - scope level – sub (whole subtree). If scope to one, means search by single level
 - timeout – preventing indefinite waiting
 - filter: – filter by objectclass (group, person, uid)
 - pageSize – manageable chunk or “pages”

Full picture of YAML definition code

```
apiVersion: v1
kind: LDAPSyncConfig
url: ldap://example.com:389
insecure: false
rfc2307:
  groupsQuery:
    baseDN: "ou=groups,dc=example,dc=com"
    scope: sub
    derefAliases: never
    pageSize: 0
  groupUIDAttribute: dn
  groupNameAttributes: [ cn ]
  groupMembershipAttributes: [ member ]
  usersQuery:
    baseDN: "ou=users,dc=example,dc=com"
    scope: sub
    derefAliases: never
    pageSize: 0
...output omitted...
```

Run the LDAP synchronization

- Login with cluster-role admin account, then perform sync:
\$ oc adm groups sync --sync-config=config.yaml --confirm
- For automatic synchronization:
 1. Create service account.
 2. Create role with appropriate permissions.
 3. Map service account to role.
 4. Create config map to communicate with LDAP IdP.
 5. Deploy cron job to perform the LDAP group synchronization.

1. Create service account

- Example of service account in YAML:

```
kind: ServiceAccount
apiVersion: v1
metadata:
  name: ldap-group-sync-acct
  namespace: ldap-group-sync
```

2. Create role with appropriate permissions

- Example of role definition in YAML:
- `apiGroups` with empty string points to core groups API as well as user resources in openshift
- `verbs` with all needed permissions to manage users and groups

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: ldap-group-sync-acct
rules:
  - apiGroups:
    - ""
    - user.openshift.io
  resources:
    - groups
  verbs:
    - get
    - list
    - create
    - update
```

3. Map service account to role.

- Project level role binding.
- Specify correct service account name
- Specify correct role name

kind: ClusterRoleBinding

apiVersion: rbac.authorization.k8s.io/v1

metadata:

name: ldap-group-sync-acct

subjects:

- kind: ServiceAccount

name: ldap-group-sync-acct

namespace: ldap-group-sync

roleRef:

apiGroup: rbac.authorization.k8s.io

kind: ClusterRole

name: ldap-group-sync-acct

4. Create config map to communicate with LDAP IdP

- Cluster must be configured to communicate with the LDAP IdP
- Important connection parameters:
 - **bindDN** and **bindPassword**

kind: ConfigMap

apiVersion: v1

metadata:

name: ldap-group-sync-acct

namespace: ldap-group-sync

data:

sync.yaml: |

kind: LDAPSyncConfig

apiVersion: v1

url: ldaps://1.2.3.4:389

insecure: false

bindDN:

cn=administrator,dc=example,dc=com

bindPassword:

file: "/etc/secrets/bindPassword"

ca: /etc/ldap-ca/ca.crt

rfc2307:

groupsQuery:

baseDN: "ou=groups,dc=example,dc=com"

scope: sub

...output omitted...

5. Deploy cron job to perform the LDAP group synchronization

```
kind: CronJob
apiVersion: batch/v1
metadata:
  name: ldap-group-sync-acct
  namespace: ldap-group-sync
spec:
  schedule: "*/30 * * * *"
  ...output omitted...
  containers:
    - name: ldap-group-sync
      image: "registry.redhat.io/openshift4/ose-
cli:latest"
      command:
        - "/bin/bash"
        - "-c"
        - "oc adm groups sync --sync-
config=/etc/config/sync.yaml --confirm"
```

Continue >

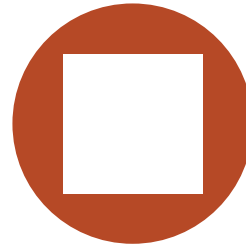
```
volumeMounts:
  ...output omitted...
volumes:
  - name: "ldap-sync-volume"
    configMap:
      name: "ldap-group-sync-acct"
  - name: "ldap-bind-password"
    secret:
      secretName: "ldap-secret"
  - name: "ldap-ca"
    configMap:
      name: "ca-config-map"
  ...output omitted...
  serviceAccountName: "ldap-group-sync-
acct"
```

Section Summary

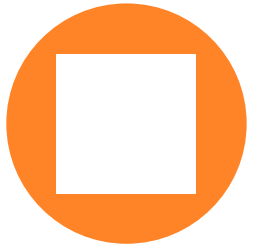
In this section, you have learned:



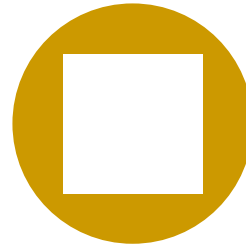
Understand LDAP



LDAP CLI Queries



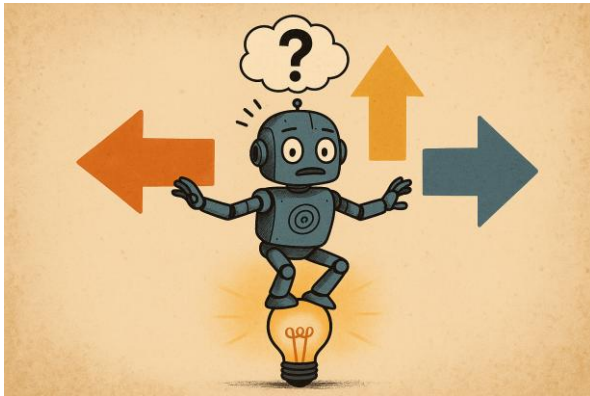
Configure OpenShift to
use LDAP directory
service



Perform Group
synchronization with
LDAP



Break / Guided Exercise?



Section 1.4

Guided Exercise

LDAP

Authentication

and Group

Synrhconization

You should be able to:

- Configure an LDAP identity provider (IdP) for RHOC.

Estimated completion time: 15-20 minutes

Section 1.5

Guided Exercise

LDAP Automate

LDAP Group

Synchronization

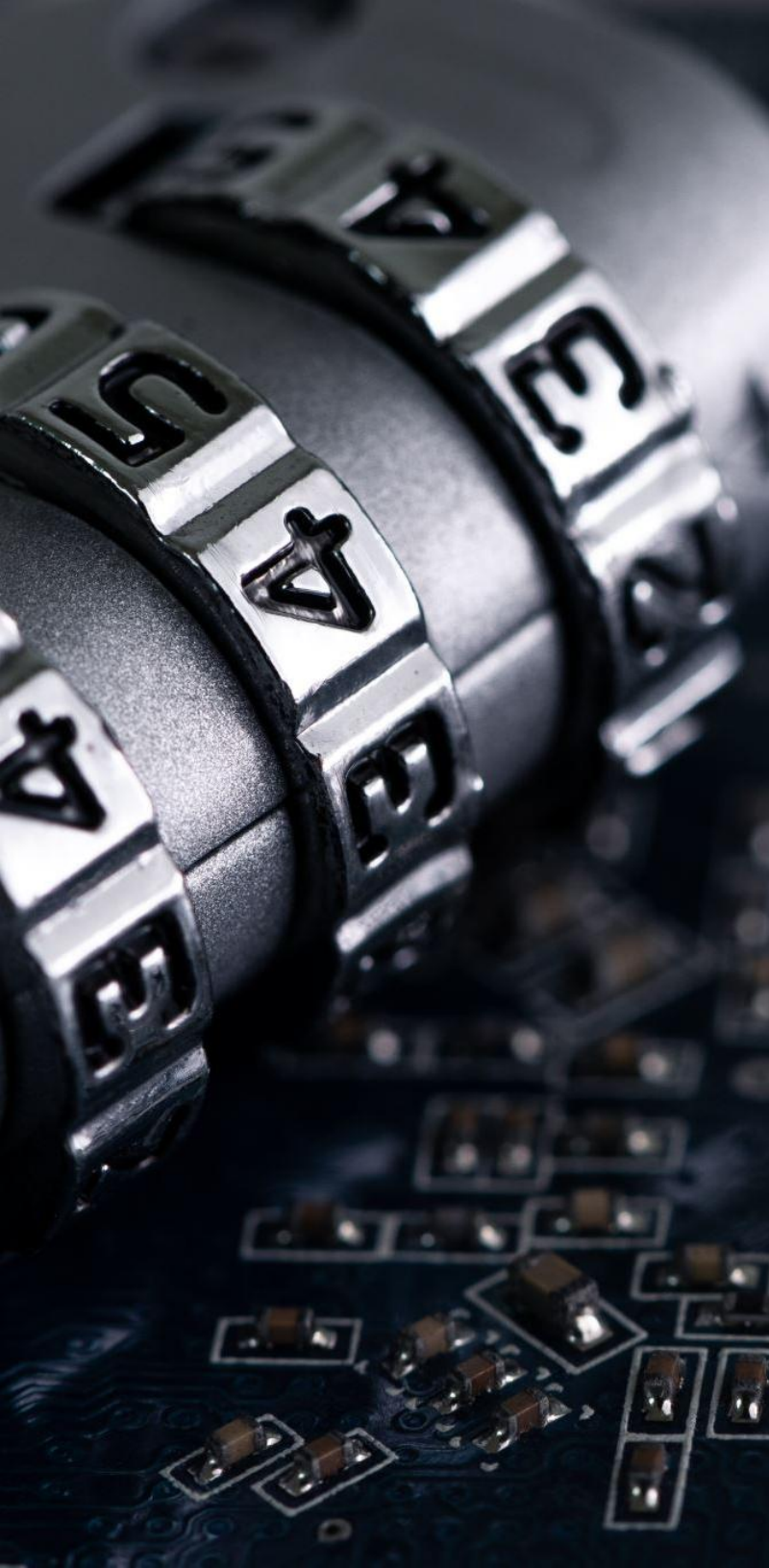
You should be able to:

- Configure an automated group synchronization for
 - the Red Hat Directory Services (RHDS) LDAP identity provider (IdP).

Estimated completion time: 10-15 minutes

OIDC Authentication and Group Claims

Chapter 1 Section 6



Objectives

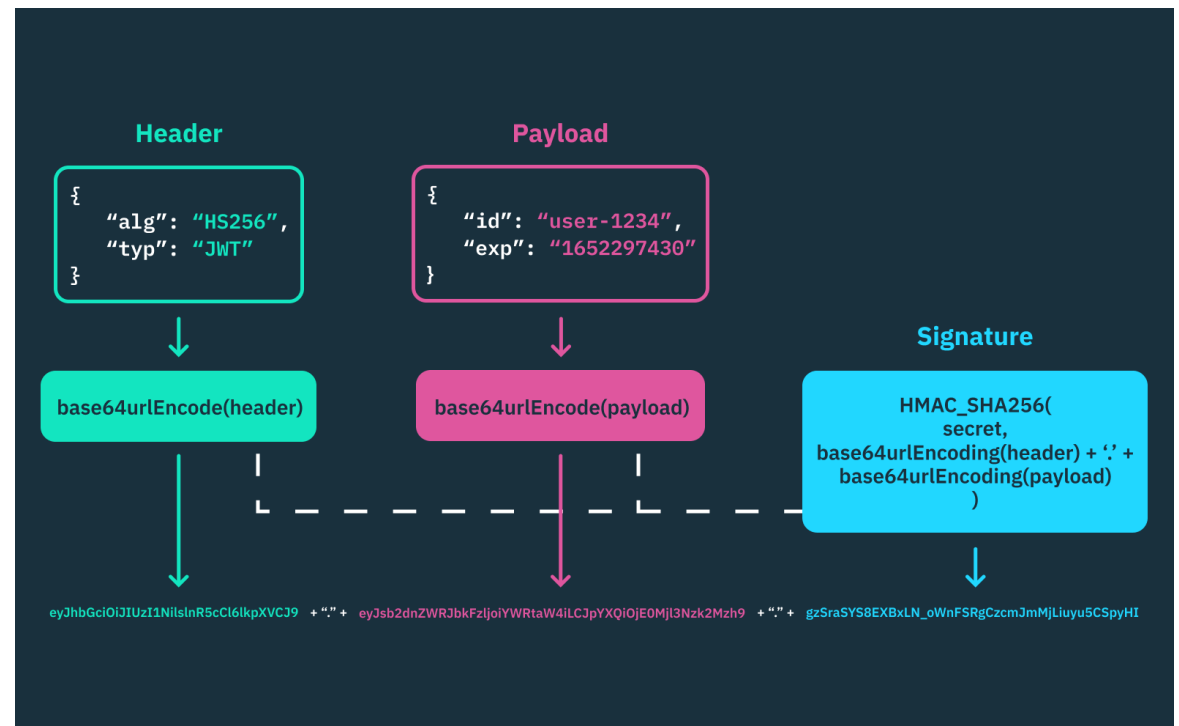
After completing this section, you will be:

- Configure an
 - **OIDC identity provider**
- Automate group synchronization
 - **between OpenShift OAuth and an OIDC server.**

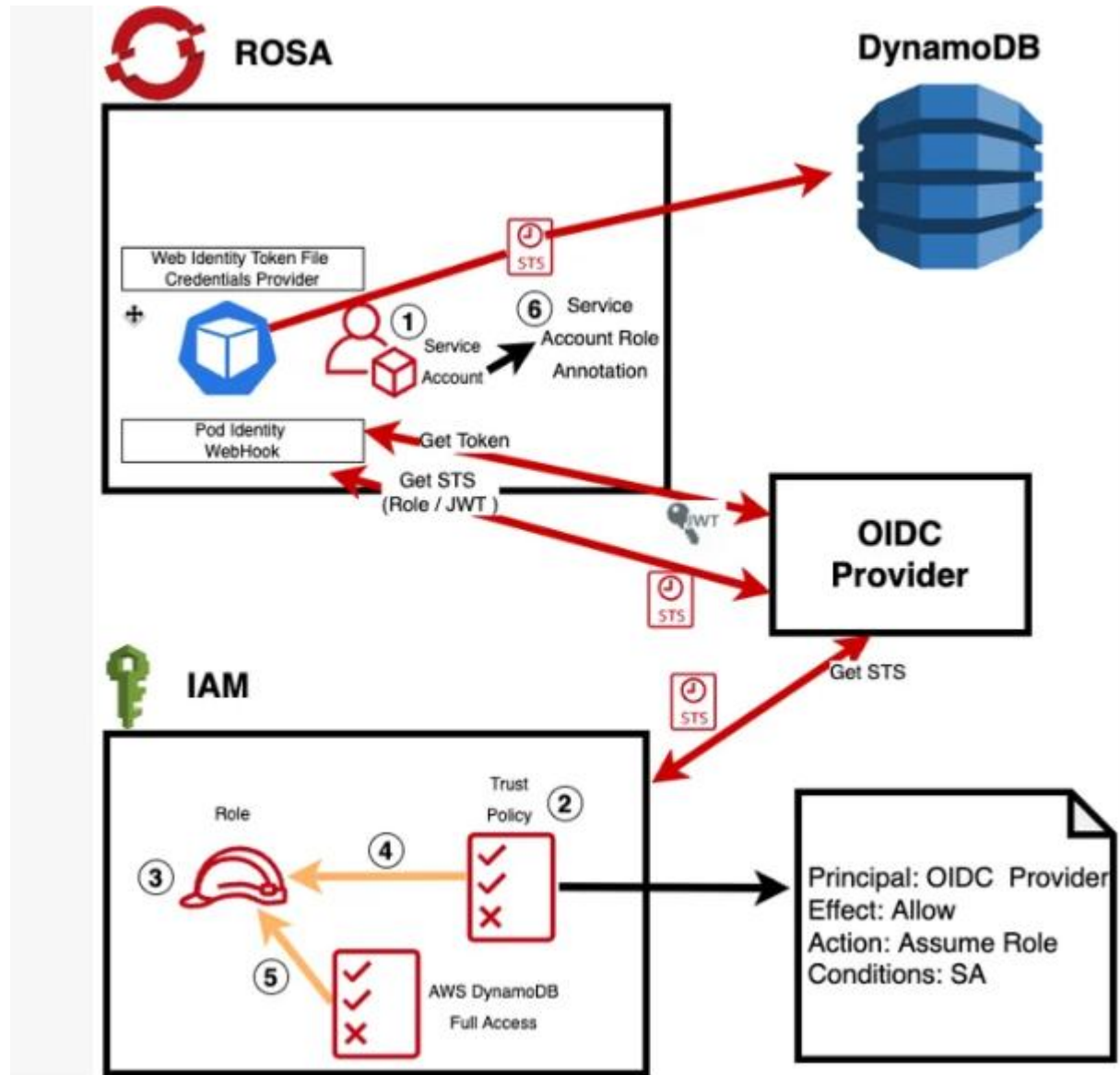
What is OpenID Connect (OIDC)?

- Set of standard protocols for delegating authentication.
- Manage and protect access to resources:
 - Cloud or on-premises data
- Verify applications or users.
- Case use:
 - Cloud authentication
 - Social authentication
 - Single Sign-On (SSO)
 - Two-factor authentication (2FA)
- Eliminate needs for
 - Setting, storing and Managing passwords locally.
- Supported OIDC with OpenShift
 - Google OIDC
 - Microsoft Identity platform
 - Keycloak
 - Red Hat SSO

- JSON Web Token (JWT)
 - JWT advantages:
 - Can parse in many programming language
 - Can propagate across networks
 - Validates messages integrity
 - JWT three parts:
 - Header
 - Payload
 - Hash signature
- 
- The diagram shows a dark blue rectangular area representing the JWT structure. At the top, the word "Header" is written in green. Below it, a rounded rectangle with a green border contains a JSON object: {"alg": "HS256", "typ": "JWT"}. A green arrow points downwards from the bottom of this rounded rectangle.



OIDC in OpenShift



Configuring OIDC IdP

1. Obtain the client ID and the client secret from the OIDC IdP client for the OpenShift integration.
2. Create an OpenShift secret object, which contains the client secret that is obtained from the OIDC client configuration, in the openshift-config namespace.
3. Create an OpenShift configuration map object, which contains the certificate authority bundle in the ca.crt file parameter, in the openshift-config namespace.
4. Create the OAuth CR YAML file to include the OIDC IdP.
5. Apply the configuration file to the OAuth CR.

Create the OAuth CR YAML file

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: oidc_provider_name
    mappingMethod: claim
    type: OpenID
    openID:
      clientID: oidc_clientid
      clientSecret:
        name: secret_name
      ca:
        name: config_map_name
Continue >
```

```
> Continue
  claims:
    preferredUsername:
      - preferred_username
      - email
    name:
      - nickname
      - given_name
      - name
    email:
      - custom_email_claim
      - email
    groups:
      - groups
  issuer: https://external_idp_url.com
```

Log in to the Cluster Through the OIDC IdP

- Using `oc login` command with `-u <username> -p <password>`

OR

- Using `oc login` command with `--token=access_token`

Obtain `access_token` from:

1. OpenShift web console
2. Help → Command line tools
3. Copy login command

Or

```
cluster-admin role$ oc get useroauthaccesstokens
```

Or

Through OpenShift REST API

OAuth Access Tokens

- Common to any IdP.
- After user logs in, OAuth server creates token
 - To authenticate to any resource API.
- OCP renew token on every authentication.
- View all OAuth access token cluster
 - Must access using cluster-admin role account

\$ oc get useroauthaccesstokens

NAME	CLIENT NAME	CREATED	EXPIRES	...
sha256~9BZ3...	openshift-...	69m	2023-06-14 13:24:42 +0000 UTC	...
sha256~lm1O...	console	75m	2023-06-14 13:19:20 +0000 UTC	...
sha256~xmpC...	openshift-...	83m	2023-06-14 13:11:24 +0000 UTC	...

Deletion and Modifications

- Remove user account in OIDC server
 - User is still logged in OpenShift.
 - User still can perform tasks in Openshift until log out.
- Modify or add parameter to user account in OIDC server
 - User is still logged in OpenShift.
 - New parameter will not take effect.
- Remove all user access tokens from OpenShift

```
$ oc delete oauthtoken $(oc get oauthtoken -o \
  jsonpath='{.items[?(@.username=="username")].metadata.name}')
```
- Deleting token will not remove identity or account information.
- User can't perform any tasks.
- User need to re-login to continue with work

IdP Mapping Methods

- Edit OAuth cluster object.
- mappingMethod parameter for the OIDC IdP as :
 - claim (non-redundant IdP) or
 - add (redundant IdP)
- Establish mappings between identity provider

Section summary

In this section, you have learned:



Configure OIDC
server



OIDC OAuth
Access token



Synchronize the
access token

Section 1.7 & 1.8 Guided Exercise

You should be able to:

- Configure Red Hat Single Sign-On (SSO)
 - as an OIDC identity provider (IdP) for OpenShift.
- Synchronize users and groups
 - from Red Hat SSO to OpenShift.
- Solve conflicts
 - when synchronizing users from more than one identity provider.

Estimated completion time: 30-45 minutes

Token and Client Certificate authentication with kubeconfig files

Chapter 1 Section 9



Objectives

After completing this section, you will be:

- Generate
 - a token or
 - a client certificate.
- Add them to a kubeconfig file.

External Client Authentication

- Authenticate external clients
 - CI/CD pipelines
 - External Monitoring tools
 - SIEM or SOAR products
- Several approach:
 - Client certificate
 - Authentication token
 - Kubeconfig

Why uses kubeconfig file?

- Manage access to Kubernetes clusters
- Allow `kubectl` and `oc` command to connect cluster
 - With correct credential.
 - Automated login.
 - Without remember username/password.
 - Copy and paste access token.
- It stores essential information:
 - Cluster API server address.
 - User authentication credentials.
 - List of contexts (users, clusters and namespaces).

Service Accounts

- Identity for processes that run in pod.
- Designed for machines, applications, and automated processes.
- Enable Pods to authenticate and interact with k8s API server.
- User accounts vs (service account)
 - Login by human users with password.
 - Intended for global.
- Service account
 - Login by application, process with keys or tokens.
 - Intended for specific namespace.
- Fundamental components of security and operational framework:
 - Works with RBAC to define fine-grained access.
 - Can be bound to ClusterRoles or “Project” Roles.

Service Account Tokens

- Service account is namespaced.
- Identical SA and SA tokens can exist on different namespace/project.
- Can be bound to
 - Pods (used for volume mounts)
 - Secrets (used for external Container registry)
 - Nodes (auto revoke token when node is removed; or re-bind when node is added)
- Older K8s version,
 - API tokens and credential automatically created.
 - Long lived. Never expired. Hence not secured
- Newer/future k8s version
 - Use TokenRequest API



The workflow

- Create new service account

```
$ oc create SA -n project1 my-new-sa
```

- Create a token

```
$ oc create token my-new-sa -n project1
```

- Use the newly created token as bearer for authentication to k8s API.
- You can set lifetime for SA token using `--duration`

```
$ oc create token my-new-sa -n project1 --duration 120m
```

```
eyJhbGciOiJSUzI1NiIsImtpZCI6IjVhJNEtybWdrMmhEZzVpcm1oWWg1MHE0R3JhZVhfdz  
N0Q2NER1NZeFAzVG8ifQ.eyJhdWQiOiJsiaHR0cHM6Ly9rdWJlcm5ldGVzLmRlZmF1bHQ  
...output omitted...
```

- Embed token in application

Service account creation on every new project

Builder

- Allowing pods to build and push pods to internal image registry.
- Bound to system:image-builder role.

Deployer

- Allowing pods to view and modify replication controllers and pods.
- Bound to system:deployer role.

Default

- Allowing pods to pull images from any image stream within internal image registry.
 - Bound to system:image-puller role
- ✓ Simply bind these SA to any cluster or project role to grant newly created pods with additional permissions.

Manage an SA

- Use `oc create sa` command
- Use `oc delete sa` command
- Use `oc get/describe sa` command
- Use `oc adm policy` command to:
 - grant permissions to SA or
 - revoke permissions from SA
- By default, every newly created SA, is automatically added as member to:
 - `system:serviceaccounts`
Includes all SA in cluster
 - `system:serviceaccounts:project`
Includes all SA in specified project

Client Certificate Authentication

- Using TLS client certificates
- OpenShift has built-in / internal Certificate Authority (CA)
 - Manages digital certificates
 - Provides trusted source for generating X.509 certificates
- X.509 certificates enables:
 - Secure communication
 - Authentication
 - Encrypt data within OpenShift cluster
- OpenShift is preconfigured to trust client certificates that internal CA signs.



Read more on <https://access.redhat.com/solutions/6054271> related to configuring additional client CA for Kubernetes API Server.

Considerations

- Use client certificates for
 - CI/CD pipeline integrations
 - Automation playbooks
 - Monitoring Tools
- Present cert and authorized clients access to API server from outside cluster.
- Between service account and client certificates:
 - Red Hat recommends using service account
 - Run automation outside cluster.
 - Reason: certs cannot be revoked in OpenShift.

Create Client Certificate

1. Create a certificate signing request (CSR).
2. Create CSR YAML resource manifest.
3. Submit CSR to Kubernetes API Server.
4. Review and approve CSR.
5. Retrieve client certificate.
6. Distribute client certificate to external application.

1. Create a certificate signing request (CSR)

- Use openssl command
 - ❖ Open source tool
 - ❖ Part of OpenSSL cryptography library
 - ❖ Various tasks related to TLS, SSL and general cryptographic operations
 - Create CSR with client information and public key
- ```
$ openssl req -nodes -newkey rsa:4096 -keyout dir/key_name \
-subj "/O=org_name/O=group_name/CN=username" -out csr_filename
```



## 2. Create the CSR YAML resource manifest

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
 name: csr_name
spec:
 signerName: kubernetes.io/kube-apiserver-client
 expirationSeconds: 604800 # one week
 request: $(base64 -w0 csr_filename)
 usages:
 - client auth
```

- OCP uses kube-apiserver-client as built-in signer of certificates .
- expirationSeconds: set certification expiration in seconds.
- Points to the csr\_filename generated in previous step.
- Specifies use case. client auth indicates that the cert is intended for client authentication purposes only.

# Create Client Certificate - remaining steps

3. Submit the CSR to k8s API server for approval

```
$ oc apply -f csr.yaml
```

4. Review and approve the CSR. Internal CA will sign and generate cert from the CSR.

```
$ oc adm certificate approve csr_name
```

5. Retrieve the client certificate: After approval and signed by internal CA.

```
$ oc get csr csr_name -o jsonpath='{.status.certificate}' \
| base64 -d > certificate_filename
```

6. Share and distribute cert: together with private key that generates the CSR, enabling external applications or client to authenticate to OpenShift cluster.

# The kubeconfig file

What and why is kubeconfig?

- Stores tokens and client certificates
- Profile to setup `kubectl` or `oc` command
- Used by most Kubernetes client libraries
- Authenticate external applications to cluster

When kubeconfig used?

1. Use `oc/kubectl` command with `--kubeconfig` option
  2. Set `KUBECONFIG` environment variable
  3. Use `~/.kube/config`
- `~/.kube/config` is generated automatically.

# Get some details from kubeconfig file

\$ oc config view

```
apiVersion: v1
clusters:
- cluster:
 server: https://api.ocp4.example.com:6443
 name: api-ocp4-example-com:6443
contexts:
- context:
 cluster: api-ocp4-example-com:6443
 user: developer/api-ocp4-example-com:6443
 name: /api-ocp4-example-com:6443/developer
- context:
 cluster: api-ocp4-example-com:6443
 namespace: default
 user: admin/api-ocp4-example-com:6443
..output omitted...
```

# Configure CLI Profiles

---

## Set Cluster

- Set server IP or CA files to be used.
- Set cluster entry in kubeconfig file.

## Set Credentials

- Set user parameters: username, password or token.
- Set user entry in kubeconfig file.

## Set Contexts

- Set default project / namespace.
- Set combination of credential and cluster.
- Set context entry in kubeconfig file.

# User Impersonation

- Enable user to act on behalf of another user
- Enable service account to act on behalf of another service account.
- Like SUDO in Linux.
- Case use:
  - Allow regular user to perform administrative tasks.
  - Debug RBAC rule by impersonating.

A decorative graphic in the top right corner featuring a cluster of colorful balloons (white, gold, red, blue, and teal) with yellow stars and confetti falling from them against a dark brown background.

# END OF DAY

Thank you for today. Safe journey home.